

GAN and Conditional GAN for Plant Leaf Synthesis on PlantVillage

Course: AI Applications

Assignment: Generative Modeling (GAN + cGAN)

Author: yousef elsayed lasheen **Date:** April 21, 2026

Executive Summary

This project implements a complete generative modeling pipeline in PyTorch to synthesize plant leaf images using the PlantVillage dataset. Two systems were developed:

1. A standard GAN to learn the leaf image distribution for a selected plant family (Apple or Tomato).
2. A Conditional GAN (cGAN) that can generate images conditioned on leaf health state (Healthy vs Diseased).

The implementation is modular and split into data handling, model architecture, training routines, visualization utilities, and inference scripts. The pipeline includes targeted preprocessing, controlled optimization settings for GAN stability (LR ratio tuning), scheduled image checkpoints, and qualitative evolution analysis from early to later training.

1. Problem Statement and Objectives

Plant disease analysis often suffers from limited balanced training data. Synthetic image generation can support augmentation, robustness, and controlled sample creation.

Main Objective

Develop a robust GAN/cGAN workflow that can:

1. Learn realistic leaf textures and structures from PlantVillage.
2. Track training dynamics and detect instability or mode collapse.
3. Support conditional generation for clinically meaningful categories (Healthy/Diseased).

Task-Wise Objectives

1. Task 1: Dataset Exploration

- Load PlantVillage and filter classes for either Apple or Tomato.
- Resize all images to 64x64 and normalize for stable GAN training.
- Visualize exactly 10 real preprocessed samples.

2. **Task 2: Learning Rate Ratio Tuning**
 - Implement DCGAN-style Generator and Discriminator.
 - Set optimizer learning rates to:
 - LR_DISC = 0.0004
 - LR_GEN = 0.0001
 - Add explicit epoch-10 observation to inspect mode collapse risk.
 3. **Task 3: Visualizing Evolution**
 - Save generated image grids every 5 epochs.
 - Compare generated outputs from Epoch 1 vs Epoch 50.
 4. **Task 4: Conditional GAN (Advanced)**
 - Extend GAN to cGAN with label embeddings in both Generator and Discriminator.
 - Enable controlled generation for Healthy or Diseased classes.
-

2. Environment and Dependencies

Installation

```
pip install torch torchvision matplotlib pillow numpy tqdm
```

requirements.txt

```
torch>=2.2.0
torchvision>=0.17.0
matplotlib>=3.8.0
pillow>=10.0.0
numpy>=1.26.0
tqdm>=4.66.0
```

Hardware/Runtime Notes

- GPU acceleration is automatically used when CUDA is available.
 - CPU fallback is supported.
 - Reproducibility is improved through random seed control.
-

3. Dataset and Preprocessing Pipeline

Dataset Source

- **PlantVillage** organized as class subfolders.
- Compatible with `torchvision.datasets.ImageFolder`.

Class Filtering Strategy

The pipeline filters classes by plant family name (**Apple** or **Tomato**) using folder name matching. This preserves all disease subclasses for the selected family.

Health Condition Mapping for cGAN

A wrapper dataset derives binary condition labels from class names:

- 0 -> Healthy
- 1 -> Diseased

Label assignment rule: class names containing **healthy** are mapped to Healthy; otherwise Diseased.

Transformations

Applied in sequence:

1. Resize to exactly 64x64
2. Random horizontal flip ($p=0.5$)
3. Convert to tensor
4. Normalize each RGB channel to $[-1, 1]$ using mean/std (0.5, 0.5, 0.5)

Visualization Utility

A dedicated utility enforces and plots exactly 10 real samples in a 2x5 grid, after denormalization to displayable RGB.

4. Model Architecture

4.1 Standard GAN (DCGAN-style)

Generator

- Input: latent vector **z** with shape (**z_dim**, 1, 1)
- Stack of transposed convolutions with batch normalization and ReLU
- Output: 3x64x64 RGB image through **Tanh**

Discriminator

- Input: 3x64x64 image
- Stack of strided convolutions with batch normalization and LeakyReLU
- Output: scalar probability via **Sigmoid**

Initialization

Weights are initialized with normal distributions (DCGAN convention):

- Convolution layers: mean 0, std 0.02
- BatchNorm weights: mean 1, std 0.02; bias 0

4.2 Conditional GAN (cGAN)

Conditional Generator

- Learns an embedding for class labels (**Healthy** / **Diseased**).
- Concatenates label embedding with latent noise channel-wise.
- Generates condition-aware synthetic leaf images.

Conditional Discriminator

- Learns a spatial label embedding reshaped to **1x64x64**.
- Concatenates this conditioning map with image channels.
- Judges realism under the provided condition.

This allows class-guided synthesis rather than unconditional generation.

5. Training Strategy and Hyperparameter Design

Loss Function

- Binary Cross Entropy (BCELoss) for adversarial objective.

Optimizers

- Adam for both networks with `betas=(0.5, 0.999)`.

Learning-Rate Ratio (Task Requirement)

- Discriminator: `LR_DISC = 0.0004`
- Generator: `LR_GEN = 0.0001`

Rationale: strengthening the discriminator update pace can improve gradient signal quality for the generator in early training, but requires monitoring to avoid overpowering.

Batch and Epoch Setup

- Default batch size: 64
- Default epochs: 50
- Latent dimension: 100

Mode Collapse Checkpoint

At **Epoch 10**, the training loop emits an explicit message requesting diversity inspection of generated outputs.

Interpretation guideline:

- High diversity (shapes/colors vary): healthy progression.
 - Near-identical outputs: potential mode collapse.
-

6. Tracking, Logging, and Visual Evolution

Snapshot Policy

Generated sample grids are saved:

- At epoch 1
- Every 5 epochs thereafter (5, 10, ..., 50)

This yields a consistent chronology of generative quality.

Comparison Script (Epoch 1 vs Epoch 50)

A utility displays side-by-side grids to show:

- **Epoch 1:** mostly unstructured noise
- **Epoch 50:** emerging leaf-like structure, texture, and color consistency

Output Directories

- Standard GAN: `outputs/gan`
 - Conditional GAN: `outputs/cgan`
-

7. Conditional Inference: Asking for Specific Leaf Types

A dedicated inference function accepts:

- `leaf_type = "healthy" or "diseased"`
- Number of requested samples

It then:

1. Encodes the requested condition label.
2. Samples latent noise.
3. Generates condition-specific synthetic images.
4. Saves and visualizes the resulting grid.

This demonstrates practical controllability of the cGAN.

8. Implementation Organization in Notebook

The notebook is organized into clear blocks:

1. Project overview and requirements
2. Imports, seed setup, and device selection
3. Task 1: data classes, filtering, transforms, dataloaders, real-sample visualization
4. Task 2: GAN models and initialization
5. Task 2/3: GAN training loop, snapshot saving, epoch comparison
6. Task 4: cGAN models, cGAN training, condition-based inference
7. Configuration and runnable execution cells

This structure supports readability, maintenance, and incremental experimentation.

9. Results Section (Fill After Running)

9.1 Dataset Overview

- Selected family: [Tomato]
- Number of filtered samples: [16011]
- Input resolution: 64x64

9.2 Visual Quality Progression

- Epoch 1 observations: [noise characteristics]
- Epoch 50 observations: [structure realism]

9.3 cGAN Conditional Quality

- Healthy generation quality: [very good]
- Diseased generation quality: [varia]
- Condition separability: [moderate]

10. Discussion

Strengths

1. Clean modular design with reusable components.
2. Full task coverage from baseline GAN to conditional generation.
3. Built-in monitoring for instability and visual progress.
4. Practical inference endpoint for controllable generation.

11. Reproducibility Checklist

- ☑ Fixed random seed function
 - ☑ Explicit dependency list
 - ☑ Deterministic preprocessing dimensions (64x64)
 - ☑ Defined architecture and initialization
 - ☑ Documented optimizer settings and hyperparameters
 - ☑ Saved intermediate outputs for verification
 - ☑ Provided inference script for deployment-style usage
-

12. Conclusion

The project successfully delivers a complete and professional GAN-to-cGAN workflow for synthetic plant leaf generation on PlantVillage. All assignment tasks were implemented: filtered data exploration, required learning-rate tuning, temporal output tracking, and condition-guided synthesis with embeddings. The pipeline is organized, extensible, and suitable as a strong base for further research-grade improvements in agricultural AI data generation.

Appendix A: Key Runtime Commands

```
# Install dependencies
pip install -r requirements.txt

# Open and run notebook sections
# Task4.ipynb
```

Appendix B: What to Include in Final Submission Package

1. Task4.ipynb
2. Task4_Comprehensive_Report.pdf
3. outputs/gan sample snapshots
4. outputs/cgan sample snapshots